

Bubble Sort

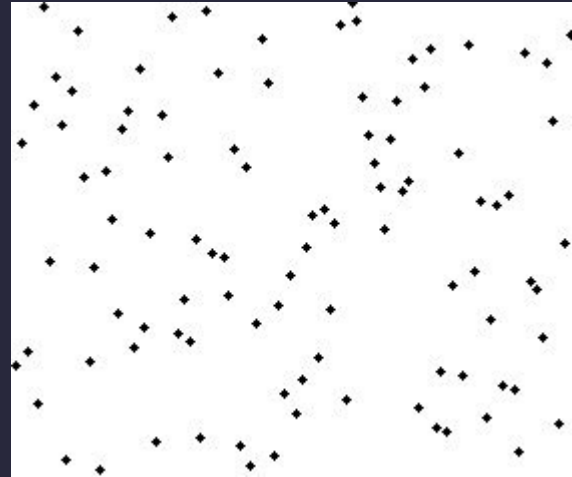
ALGORITMO DE ORDENAÇÃO

Resolução de Problemas
Estruturados em Computação



/CONCEITO GERAL

O **bubble sort**, ou ordenação por flutuação (literalmente "por bolha"), é um algoritmo de ordenação dos mais simples. A ideia é percorrer um conjunto de elementos diversas vezes, e a cada passagem fazer flutuar para o topo o maior elemento da sequência. Essa movimentação lembra a forma como as bolhas em um tanque de água procuram seu próprio nível, e disso vem o nome do algoritmo.



/INSERINDO ELEMENTOS

[0]	[1]	[2]	[3]	[4]
7	2	12	8	3

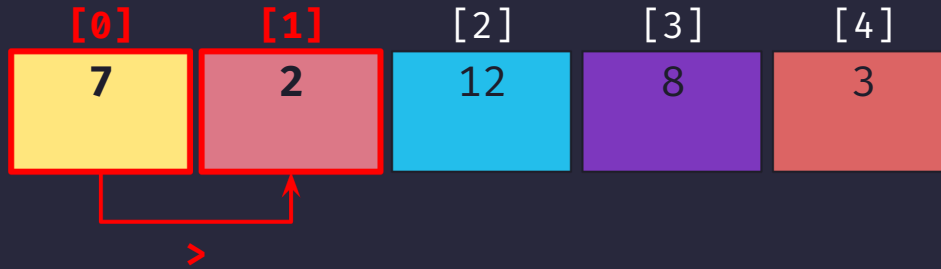
Nosso primeiro objetivo é
colocar o 12 na posição [4].

/ORDENANDO ELEMENTOS



Vamos analisar o primeiro par,
posição [0] e [1].

/ORDENANDO ELEMENTOS



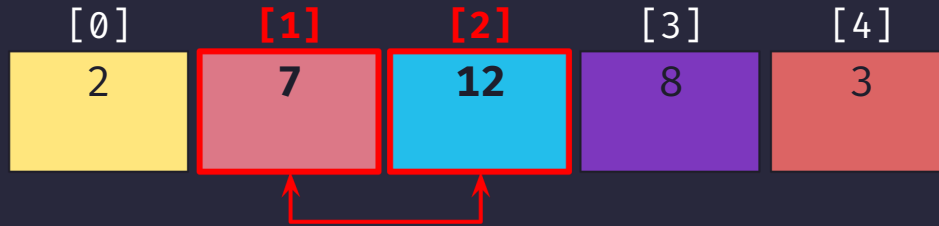
O elemento da posição [0] é maior que o elemento da posição [1]? ($7 > 2$)

/ORDENANDO ELEMENTOS



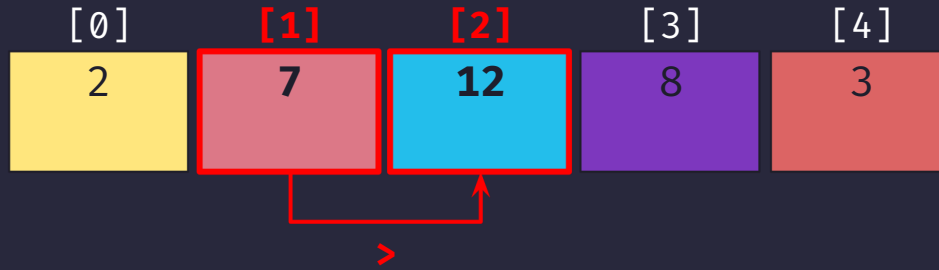
Trocamos o valor da posição [0] pelo valor da posição [1].

/ORDENANDO ELEMENTOS



Agora vamos para o próximo par. Posição [1] e [2].

/ORDENANDO ELEMENTOS



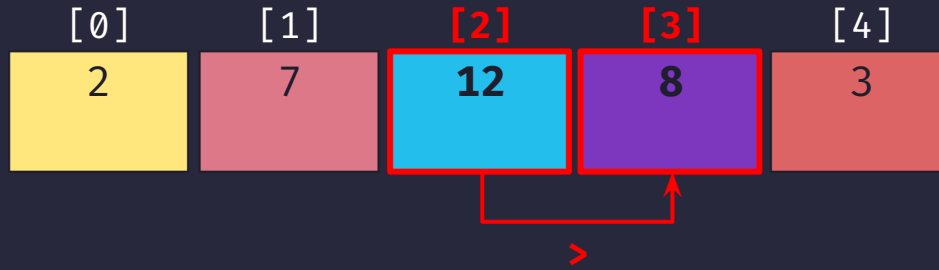
O elemento da posição [1] é maior que o elemento da posição [2]? ($7 > 12$)

/ORDENANDO ELEMENTOS



Então mantemos os dois
elementos nas mesmas posições.

/ORDENANDO ELEMENTOS



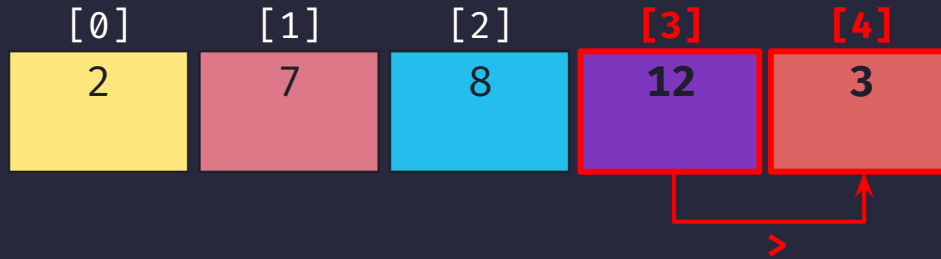
O elemento da posição [2] é maior que o elemento da posição [3]? ($12 > 8$)

/ORDENANDO ELEMENTOS



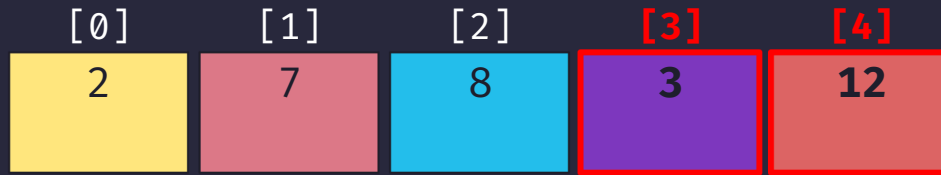
Trocamos o valor da posição [2] pelo valor da posição [3].

/ORDENANDO ELEMENTOS



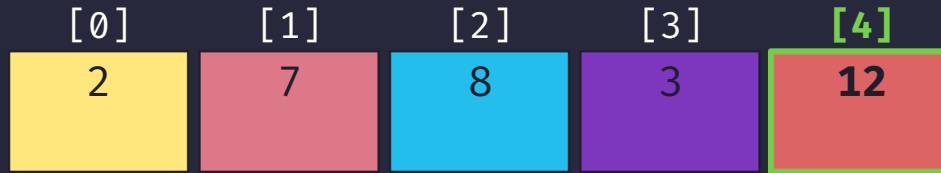
O elemento da posição [3] é maior que o elemento da posição [4]? ($12 > 3$)

/ORDENANDO ELEMENTOS



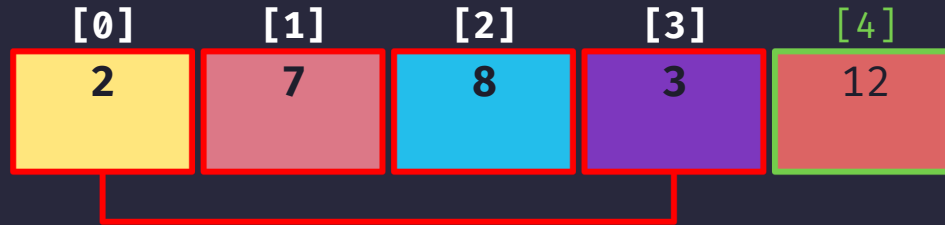
Trocamos o valor da posição [3] pelo valor da posição [4].

/ORDENANDO ELEMENTOS



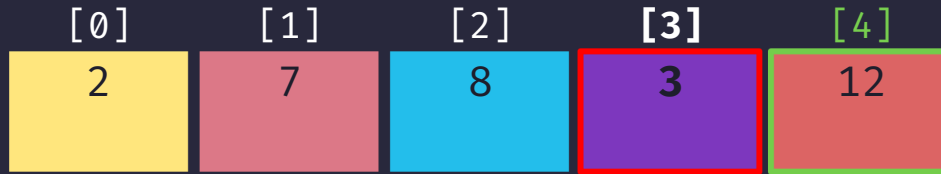
Com isso, concluímos o objetivo de colocar o 12 na última posição ([4]).

/ORDENANDO ELEMENTOS



Agora vamos ignorar o valor 12 que já está na posição correta e colocar o 8 na posição [3].

/ORDENANDO ELEMENTOS



Percebam que agora vamos considerar a posição [3] como a última posição, pois não iremos iterar pela posição [4].

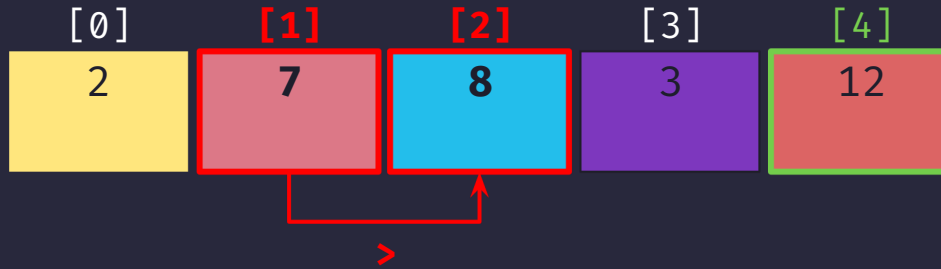
/ORDENANDO ELEMENTOS



O elemento da posição [0] é maior que o elemento da posição [1]? ($2 > 7$)

Mantemos os dois elementos nas mesmas posições.

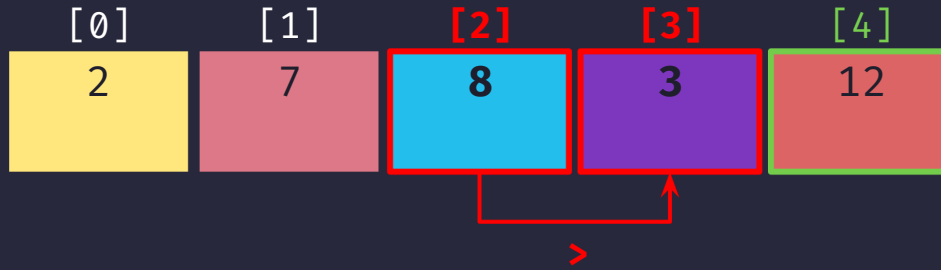
/ORDENANDO ELEMENTOS



O elemento da posição [1] é maior que o elemento da posição [2]? ($7 > 8$)

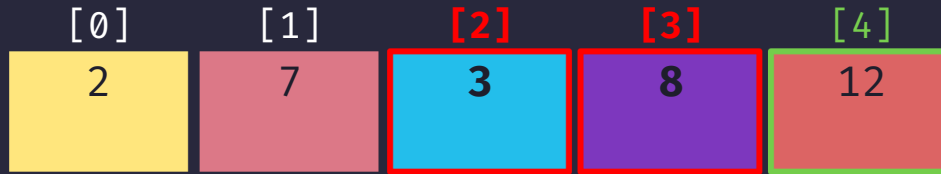
Mantemos os dois elementos nas mesmas posições.

/ORDENANDO ELEMENTOS



O elemento da posição [2] é maior que o elemento da posição [3]? ($8 > 3$)

/ORDENANDO ELEMENTOS



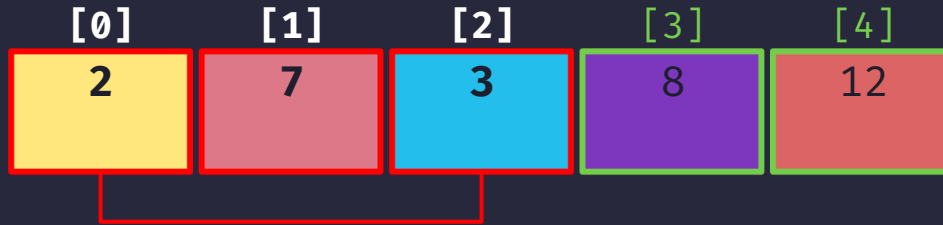
Trocamos o valor da posição [2] pelo valor da posição [3].

/ORDENANDO ELEMENTOS



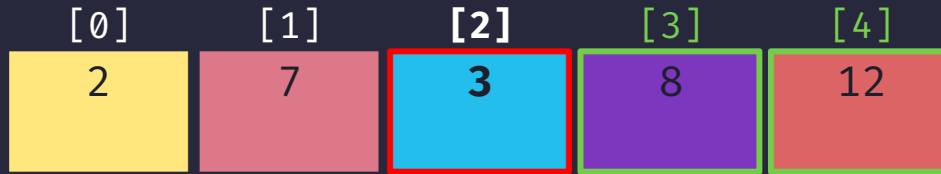
Com isso, concluímos o objetivo de colocar o 8 na posição [3].

/ORDENANDO ELEMENTOS



Agora vamos ignorar o valor 8 e 12 que já estão nas posições corretas e colocar o 7 na posição [2].

/ORDENANDO ELEMENTOS



Percebam que agora vamos considerar a posição [2] como a última posição, pois não iremos iterar pelas posições [3] e [4].

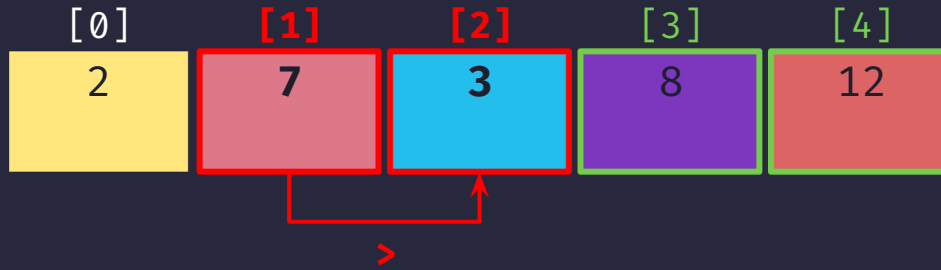
/ORDENANDO ELEMENTOS



O elemento da posição [0] é maior que o elemento da posição [1]? ($2 > 7$)

Mantemos os dois elementos nas mesmas posições.

/ORDENANDO ELEMENTOS



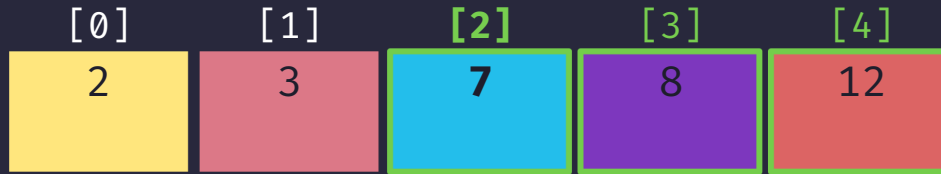
O elemento da posição [1] é maior que o elemento da posição [2]? ($7 > 3$)

/ORDENANDO ELEMENTOS



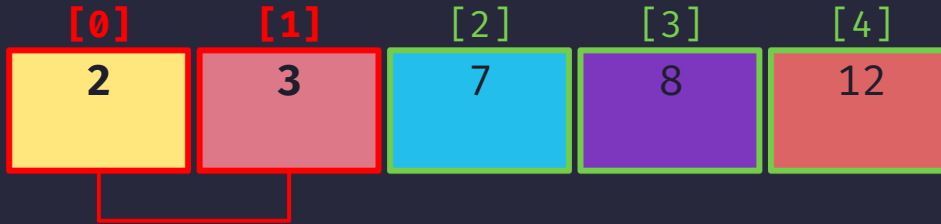
Trocamos o valor da posição [1] pelo valor da posição [2].

/ORDENANDO ELEMENTOS



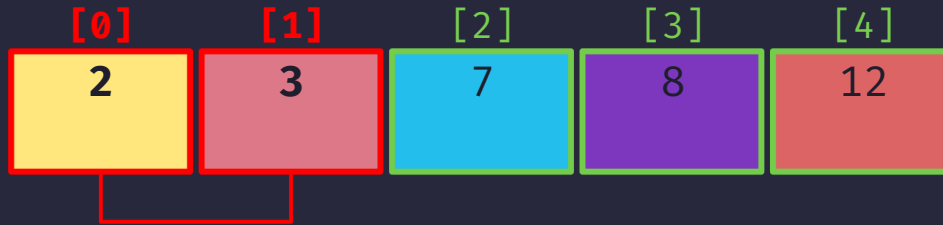
Com isso, concluimos o objetivo de colocar o 7 na posição [2].

/ORDENANDO ELEMENTOS



Agora vamos ignorar os valores 7, 8 e 12 que já estão nas posições corretas e colocar o 3 na posição [1].

/ORDENANDO ELEMENTOS



O elemento da posição [0] é maior que o elemento da posição [1]? ($2 > 3$)

/ORDENANDO ELEMENTOS

[0]	[1]	[2]	[3]	[4]
2	3	7	8	12

Então significa que todos os
nossos valores já estão
ordenados corretamente.

/DICAS ÚTEIS

- Na primeira iteração o maior valor será colocado na última posição. Para a próxima iteração, iremos desconsiderar a posição onde esse valor foi inserido, pois ele já está na posição correta. Ou seja, toda vez que o maior valor estiver na última posição, passamos a desconsiderá-la.
- A cada iteração, o tamanho do array que será iterado reduz em 1 gradativamente.
- Durante a iteração, alguns valores já são reordenados (troca de valores) para reduzir a quantidade de iterações que serão executadas. Isso faz com que a nossa ordenação fique mais otimizada.
- Funções que podem facilitar a implementação do algoritmo:
 - ♦ `swap(a, b)` - troca os valores
 - ♦ `bubbleSort(value, size)` - função recursiva com a lógica do algoritmo

